

ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

ALLIANCE UNIVERSITY

Alliance School of Advanced Computing

Student Name: Afghan Abrar Ali Khan

Register Number: PROV/ASAC/MCA/7/26/017

Project Name: Demand Forecasting for a Quick-Commerce Store Using Machine Learning

ACADEMIC YEAR: 2026-2028

Department of Computer Applications

Program - Master of Computer Applications

Semester 1



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me throughout the successful completion of this project.

I am thankful to my Project Guide for providing valuable guidance, suggestions, encouragement, and continuous support throughout the development of this project. I also express my gratitude to the faculty members and my institution for providing the necessary resources and a supportive learning environment.

I would also like to thank my friends and classmates for their cooperation and support during the completion of this project. Finally, I am deeply grateful to my family for their constant encouragement, motivation, and support.

I sincerely thank everyone who contributed directly or indirectly to the successful completion of this project.

Student Name: Afghan Abrar Ali Khan

Register Number: PROV/ASAC/MCA/7/26/017

Project Title: Demand Forecasting for a Quick-Commerce Store Using Machine Learning

Signature of Student

Signature of Faculty & Project Guide

Project Faculty & Project Guide Name: Oleti Durga Bravish

1.ABSTRACT

Quick-commerce services promise to deliver frequently purchased products within a short time. To meet this promise, a store must keep the right products available at the right time. Demand changes according to product type, day of the week, calendar events, price, customer routines, and other external factors. If demand is underestimated, customers may face stock-outs. If it is overestimated, the store may hold excess inventory and increase storage or wastage costs. This project, "Demand Forecasting for a Quick-Commerce Store Using Machine Learning," explores the use of historical sales data and machine-learning methods to estimate product demand and present the results through interactive Power BI dashboards.

The project workflow includes data loading, transformation from a wide sales format into a date-based structure, joining calendar and selling-price information, handling missing values, feature engineering, model training, evaluation, and visualization. The tested models are Linear Regression, Random Forest, and XGBoost. Their performance is compared using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). In the experiment recorded for this project, Linear Regression produced the lowest MAE, while XGBoost produced the lowest RMSE and the highest R^2 among the three tested models. The scores indicate that the task remains challenging and that further feature engineering and validation are required before operational use.

Power BI is used to communicate demand patterns, actual-versus-predicted demand, product-level differences, and inventory-related indicators. The project demonstrates an end-to-end beginner-level forecasting workflow and a practical way to turn model outputs into information that can support inventory planning. The results are experimental and should not be treated as a production-ready replenishment system without additional validation.

Keywords: quick-commerce, demand forecasting, machine learning, XGBoost, Linear Regression, Random Forest, inventory optimization, Power BI.

2. INTRODUCTION

2.1 Background

Quick-commerce is a retail model designed to deliver groceries, household essentials, personal-care products, snacks, and other everyday items in a very short period. Unlike traditional retail, where customers may plan a weekly visit, quick-commerce customers often expect immediate availability and rapid delivery. This creates a difficult operational requirement: the store must maintain sufficient inventory while limiting the cost of keeping too much stock.

Demand forecasting estimates how much of a product customers are likely to purchase during a future period. A useful forecast gives store teams a quantitative starting point for replenishment, workforce planning, and inventory allocation. Forecasting is not simply a matter of calculating an average. Demand can change across weekdays, months, seasons, promotions, events, product categories, and locations. Some items have steady demand, while others experience irregular or intermittent purchases.

Machine learning provides methods that can learn relationships between historical demand and explanatory features. Linear Regression provides a relatively simple baseline. Random Forest can represent nonlinear relationships by combining decision trees. XGBoost builds an ensemble of boosted trees and can model complex feature interactions. Comparing several methods helps

determine which approach performs better on a particular dataset rather than assuming that the most complex algorithm is always best.

2.2 Motivation

The motivation for this project is the gap between raw sales records and decisions that store managers need to make. A dataset may contain thousands of product-day observations, but managers need concise answers: which items are selling more, how does demand differ by weekday, how close are forecasts to actual demand, and where might stock planning require attention? A machine-learning model can produce estimates, while a dashboard can make those estimates easier to interpret.

This project also provides an opportunity to understand the complete analytics lifecycle. It connects data preparation, feature creation, model evaluation, and business reporting in a single workflow. The project is implemented with beginner-friendly tools and documented so that the process can be repeated and improved.

2.3 Aim and Objectives

The main aim is to develop and evaluate a machine-learning-based workflow for forecasting product demand in a quick-commerce setting and to present useful findings through interactive dashboards.

- Load and understand historical sales, calendar, and selling-price data.
- Transform sales records into a structured date-product dataset suitable for analysis.
- Clean missing values and create time-based and lag-based features.
- Train and compare Linear Regression, Random Forest, and XGBoost models.
- Evaluate the models using MAE, RMSE, and R^2.

- Visualize demand patterns and forecast comparisons in Power BI.
- Discuss how forecasts can inform inventory planning while acknowledging model limitations.

2.4 Scope of the Project

The scope covers historical demand analysis, basic feature engineering, comparison of three regression models, evaluation on a held-out portion of the prepared data, and dashboard-based reporting. The available working export used for the current Power BI report contains five HOBBIES products and one store, CA_1. Therefore, the dashboards demonstrate the analysis workflow for this selected subset rather than representing every product or store in the original source data.

The project does not automatically place purchase orders, connect to a live inventory system, or guarantee stock availability. Weather, local purchasing behaviour, promotions, and customer-level signals were identified as potentially useful variables in the broader problem statement, but the implemented model should only be described as using the features actually prepared in the notebook and exported dataset. Any future expansion should add such external features only after reliable data becomes available.

2.5 Organization of the Report

The report introduces the business problem, reviews the limitations of a manual or basic forecasting approach, describes the proposed workflow and dataset, explains the technologies and algorithms, presents the data flow and experimental outputs, discusses the Power BI dashboards, and concludes with limitations and future improvements. Appendices provide metric definitions and basic execution notes.

3. PROBLEM STATEMENT

3.1 Problem Definition

Quick-commerce inventory decisions must be made with limited time and incomplete knowledge of future demand. Historical sales contain patterns, but these patterns are not always easy to identify by manual inspection. A store that uses only a fixed average or personal judgement may fail to respond to changes in weekday demand, calendar events, product popularity, and recent sales trends.

The technical problem addressed in this project is to estimate demand for a product-day observation using historical sales and available explanatory features, then evaluate whether machine-learning models produce useful predictions. The business problem is to present the forecast and demand analysis in a form that can support better stock-planning discussions.

3.2 Consequences of Forecasting Errors

An under-forecast occurs when predicted demand is lower than actual demand. It can contribute to insufficient stock, missed sales, and customer dissatisfaction. An over-forecast occurs when predicted demand is higher than actual demand. It can contribute to excess inventory, tied-up working capital, storage pressure, and possible expiry or markdown costs for perishable products. These consequences are not symmetric for every item; the cost of an error depends on the product and operating context.

3.3 Research Questions

1. What demand patterns can be identified from the selected historical sales records?

2. How do Linear Regression, Random Forest, and XGBoost compare on the prepared dataset?

3. How closely do predicted values match actual demand under the selected evaluation procedure?

4. How can interactive Power BI dashboards make the results easier to interpret?

5. What additional data and validation would be needed before using the forecasts for real inventory decisions?

3.4 Success Criteria

The project is considered successful as a learning prototype if it can load and prepare the data, train more than one model, calculate standard evaluation metrics, export predictions, and communicate the results in a readable dashboard. A claim of operational success would require additional evidence, including time-based backtesting, comparison with a business baseline, assessment of stock-out and overstock costs, and validation on data from multiple products and stores.

4. EXISTING SYSTEM

4.1 Manual and Basic Forecasting Practices

In a basic retail workflow, staff may estimate future requirements from recent sales, a fixed average, past experience, or a spreadsheet. These methods are easy to understand and may be adequate for products with stable demand. However, they can be difficult to scale when there are many products and when sales behaviour changes over time. Manual methods may also produce inconsistent results when different people apply different assumptions.

4.2 Limitations

- Limited ability to model interactions among several variables at once.
- Manual review becomes time-consuming as the number of products and dates increases.

- A fixed average may not respond quickly to recent changes in demand.
- Forecast assumptions may not be documented consistently.
- Results may be distributed across spreadsheets rather than presented in a single dashboard.
- Basic approaches may not provide a consistent way to compare predicted demand with actual demand.

4.3 Why a Data-Driven Approach Is Useful

A data-driven workflow can process repeated observations consistently and quantify prediction errors. Machine learning does not remove uncertainty, but it provides a reproducible way to learn from available features and compare alternatives. Dashboards complement this process by enabling users to filter and inspect results rather than relying only on a static model output.

4.4 Existing-System Comparison

Aspect	Basic/manual approach	Proposed project approach
Data handling	Manual spreadsheet review	Programmatic transformation and merging
Forecast logic	Judgement or simple averages	Three supervised regression models compared
Evaluation	Often informal	MAE, RMSE and R^2 reported
Presentation	Separate tables or files	Interactive Power BI report pages
Limitations	Hard to scale and reproduce	Still limited by data coverage and validation

5. PROPOSED SYSTEM

5.1 Overview

The proposed system is an end-to-end analytical pipeline. Historical sales are loaded and reshaped, calendar information is joined by the date identifier, selling prices are added using the store-item-week keys, and missing values are inspected. Features are created to represent time and recent demand. The

prepared data is used to train and compare regression models. Predictions and relevant columns are exported for dashboard development.

5.2 Main Components

1. Data ingestion: read the sales, calendar, and price files.

2. Data preparation: reshape daily sales columns into rows and establish a product-date observation structure.

3. Data integration: join calendar fields and selling-price information using matching identifiers.

4. Data cleaning: inspect missing prices and other incomplete records, and apply a documented handling rule.

5. Feature engineering: derive calendar features, event indicators, lagged demand, and rolling demand summaries.

6. Model development: train Linear Regression, Random Forest, and XGBoost using a consistent target definition.

7. Evaluation: calculate MAE, RMSE, and R^2 on the evaluation sample.

8. Reporting: export actual and predicted demand and build interactive Power BI dashboards.

5.3 Expected Benefits

The proposed system makes the workflow repeatable, enables model comparison, and improves visibility of demand patterns. It can help a user identify products or periods that deserve further investigation. Forecasts may support replenishment planning, but final decisions should also consider lead time, current stock, supplier constraints, minimum order quantities, shelf life, and the cost of running out of stock.

5.4 System Requirements

Category	Requirement
Hardware	A computer capable of running Python, Jupyter Notebook, and Power BI Desktop; sufficient memory for CSV processing.
Operating system	Windows environment used during development.
Programming	Python and Jupyter Notebook through Anaconda.
Libraries	Pandas, NumPy, scikit-learn, XGBoost, and joblib where used by the project.
Visualization	Microsoft Power BI Desktop.
Input files	Historical sales, calendar, and selling-price files; prepared CSV for dashboard reporting.
Output	Model metrics, predicted-demand export, and interactive report pages.

5.5 High-Level Architecture

The architecture can be represented as: source files → data preparation → feature engineering → model training and evaluation → prediction export → Power BI dashboards → inventory-planning interpretation. Each stage has a defined input and output, making errors easier to locate and future changes easier to test.

6. ABOUT DATASET

6.1 Data Sources and Structure

The project uses historical retail sales data together with calendar and selling-price information. The sales source is provided in a wide format: identifiers describe the item, department, category, store, and state, while daily sales appear in separate columns such as d_1 through d_1913. The calendar file maps each day identifier to a date and provides calendar-related variables. The selling-price file records the price for a product in a store and week.

During the project exploration, the sales table was observed to contain 30,490 rows and 1,919 columns. The selling-price table contained approximately 6.84 million rows and four columns. These are source-table

dimensions from the working files and should not be confused with the smaller subset exported for model experiments and dashboarding.

6.2 Important Fields

Field or group	Description and use
item_id / id	Identifies a product or product-store record, depending on the source table.
store_id	Identifies the store associated with a sales or price record.
state_id	Identifies the state or region associated with a store.
d_1 … d_1913	Daily sales quantities stored in wide-format columns.
date	Calendar date corresponding to the daily observation.
wm_yr_wk	Week identifier used to connect calendar dates with selling prices.
weekday, wday, month, year	Calendar attributes used for analysis and feature engineering.
event_name_1 / event_type_1	Calendar event information when available.
sell_price	Recorded selling price for the item-store-week combination.
demand	Target quantity derived from historical sales.
lag_1 / lag_7	Previous-period demand features as defined in the preparation workflow.
rolling_7	Rolling demand summary over a seven-period window as defined in the notebook.
Predicted_Demand	Model estimate exported for comparison with actual demand.

6.3 Data Transformation

The first major transformation converts the daily sales columns into a long or tidy structure. Instead of keeping one row with hundreds or thousands of daily columns, each row represents a product-store-date observation with a demand value. This form is more convenient for time-series feature engineering, joining calendar information, model training, and visualization.

The day identifier is matched to the calendar table to obtain a real date and relevant calendar fields. The week identifier is then used with store and item identifiers to attach the appropriate selling price. The join keys must have consistent data types and matching values; otherwise, joins can create missing attributes or duplicate records.

6.4 Missing Values and Data Quality

In an intermediate merged table, the working notebook showed 9,565 rows and 19 columns, with 2,282 missing values in sell_price. After the applied preparation and filtering steps, the prepared dataset was reported as 9,530 rows and 27 columns with no missing values. These figures describe the project's intermediate and prepared subsets. Before repeating the experiment, the exact cleaning rules should be reviewed to confirm that rows were not removed in a way that introduces selection bias.

Missing selling prices should not automatically be treated as zero because a missing price is not the same as a free product. A robust workflow should inspect why values are missing, verify join keys, and document whether records are excluded, imputed, or handled by a separate rule. The chosen rule should be applied consistently during training and prediction.

6.5 Exploratory Demand Statistics

The project's prepared sample showed average demand of approximately 0.681 units, a standard deviation of approximately 1.297, and a maximum observed value of 15. The low average relative to the maximum indicates that the selected demand series contains many low-demand observations and occasional larger values. Such a distribution can make forecasting difficult: a model that predicts values near the average may look reasonable for many rows but miss the occasional spikes that matter for inventory decisions.

6.6 Feature Engineering

Feature engineering transforms raw columns into explanatory variables that may help a model learn demand patterns. Calendar features include day, day of week, week number, quarter, month, year, and event indicators. Lag features describe earlier demand values, while rolling features summarize recent demand. Price may also be useful when the data is correctly joined and has sufficient variation.

Lag and rolling features must be constructed without using future information. For example, a forecast for a given date must not use the actual demand from that same date or from later dates as an input. In a production setting, rolling calculations should be shifted appropriately and evaluated using a chronological split. These safeguards prevent data leakage and provide a more realistic estimate of future forecasting performance.

6.7 Dataset Limitations

- The dashboard export currently represents a small subset of the original source, including five HOBBIES products and one store.
- Some potentially useful external variables, such as weather and live promotion data, are not confirmed as included in the final model export.
- The prepared dataset may not represent every season or every operational scenario.
- A single set of evaluation metrics does not establish how the model performs across all products and future time periods.
- Product-level demand may be intermittent, which can reduce the usefulness of general regression metrics alone.

7. TECHNOLOGIES USED

7.1 Python

Python is the main programming language used for data preparation, feature engineering, model training, and evaluation. Its readable syntax makes it suitable for a beginner-level analytics project, and its ecosystem includes libraries for tabular data processing and machine learning.

7.2 Anaconda and Jupyter Notebook

Anaconda provides an environment for installing and managing Python packages. Jupyter Notebook supports an interactive workflow in which code,

outputs, tables, and explanations can be kept together. This is useful for documenting each project stage and checking intermediate results before moving to the next stage.

7.3 Pandas and NumPy

Pandas is used for loading CSV files, reshaping tables, merging datasets, checking missing values, creating features, and preparing exports. NumPy supports numerical operations and array-based calculations. Together, these libraries provide the data-processing foundation for the project.

7.4 Scikit-learn

Scikit-learn provides implementations of Linear Regression and Random Forest and utilities for splitting data, fitting models, generating predictions, and calculating evaluation metrics. It also offers a consistent interface that helps make model comparisons easier to organize.

7.5 XGBoost

XGBoost is a gradient-boosted tree library. It builds a sequence of trees, with later trees attempting to reduce errors made by earlier trees. It can model nonlinear relationships and feature interactions, but its performance depends on data quality, feature design, parameter choices, and evaluation methodology. It is not automatically superior to simpler models.

7.6 Microsoft Power BI

Power BI is used to build interactive dashboards from the exported analytical dataset. Visuals and slicers allow a viewer to explore demand by date, weekday, product, event, and other available fields. Dashboards make results accessible to users who may not want to read raw tables or notebook outputs.

7.7 File Formats and Supporting Tools

CSV is used for portable tabular data exports. Pickle/joblib-style model serialization may be used to store trained models for later loading in a Python application. The exact packages and versions should be recorded from the actual development environment if a reproducible software inventory is required.

7.8 Technology Summary

Technology	Role in the project
Python	Main language for the analytics pipeline.
Jupyter Notebook	Interactive development and documentation.
Pandas	Data loading, reshaping, merging, and cleaning.
NumPy	Numerical operations.
scikit-learn	Baseline model, Random Forest, splitting, and metrics.
XGBoost	Gradient-boosted tree regression model.
Power BI Desktop	Interactive dashboards and business reporting.
CSV	Data exchange between notebook and Power BI.

8. DATA FLOW DIAGRAM

8.1 Context-Level Data Flow

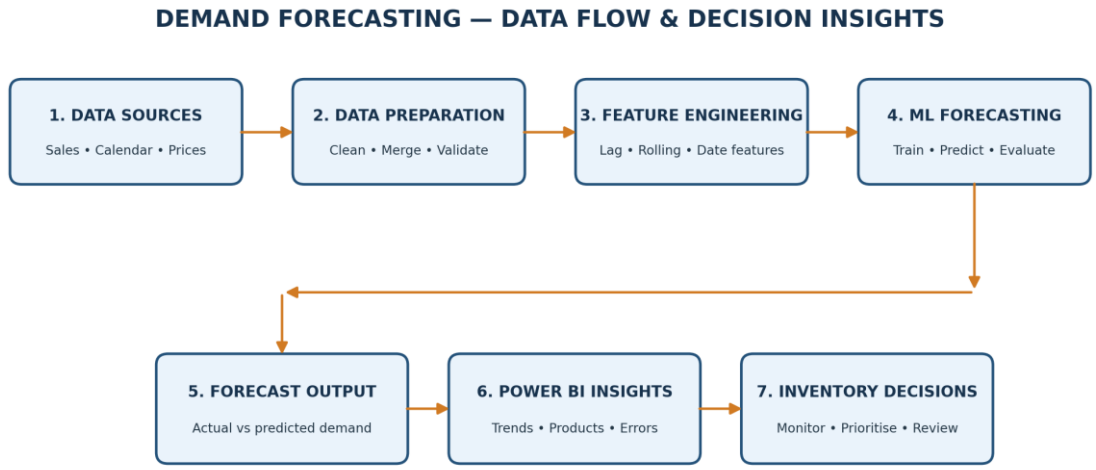
At the highest level, the system receives historical sales, calendar information, and selling prices. The data-preparation process converts the inputs into a model-ready dataset. The machine-learning process produces predictions and evaluation metrics. The reporting process combines the prepared data and predictions into dashboards for a project user or inventory planner.

Textual context diagram:

[Sales Data] ─┐
[Calendar Data] ─┼─> [Data Preparation and Feature Engineering] ─> [ML Models] ─> [Predictions and Metrics] ─> [Power BI Dashboards] ─> [Inventory Planner]

8.2 Level-1 Data Flow

Figure 7.1. Data Flow and Interactive Insight Path



Interactive insight path: select a product or date in Power BI → compare actual and predicted demand → inspect forecast error → review inventory attention.

Feedback loop: dashboard observations can guide future feature improvements and model retraining.

The diagram presents the end-to-end flow from source data to forecasting and inventory review. In the dashboard, slicers and cross-filtering allow users to select products and dates, then compare demand patterns, prediction differences, and product-level attention indicators. The diagram represents the analytical workflow; the degree of interactivity depends on the Power BI report controls configured in the actual file.

1. Process 1 — Load source data: read sales, calendar, and price tables.

2. Process 2 — Reshape sales: convert daily sales columns into date-level observations.

3. Process 3 — Integrate and clean: join calendar and price features, inspect missing values, and validate records.

4. Process 4 — Engineer features: derive date fields, event flags, lag features, and rolling summaries.

5. Process 5 — Train models: fit Linear Regression, Random Forest, and XGBoost using the selected training procedure.

6. Process 6 — Evaluate models: compare actual demand with predictions using MAE, RMSE, and R^2.

7. Process 7 — Publish analytical output: export relevant fields and build Power BI visuals.

8. Process 8 — Interpret: review demand trends and possible inventory risks; apply business judgement before making decisions.

8.3 Data Stores and Outputs

The main data stores are the original source files, the prepared model dataset, model artifacts where saved, and the prediction CSV used by Power BI. Outputs include model evaluation metrics, actual-versus-predicted comparisons, demand trend visuals, product summaries, and inventory-related indicators. Each output should be traceable to the source dataset and model version used to produce it.

8.4 Data Flow Controls

Useful controls include checking row counts before and after joins, confirming unique keys where expected, inspecting missing-value counts, ensuring that demand columns are numeric, and verifying that predictions contain no unexpected null values. Model evaluation should use a documented split, and any features based on past demand must be created without exposing the target date's actual value.

9. OUTPUT AND RESULTS

9.1 Data Preparation Output

The preparation stage produced a structured dataset with date-level observations and engineered features. The final prepared subset was reported as

9,530 rows and 27 columns with no missing values after the project's cleaning steps. A separate export was created for Power BI with actual demand, predicted demand, product and store identifiers, price, and time-related fields.

9.2 Model Evaluation

Three regression models were trained and evaluated. MAE measures the average absolute difference between actual and predicted demand. RMSE gives more weight to larger errors because the errors are squared before averaging and taking the square root. R^2 compares the model with a mean-based reference under the metric's assumptions; it can be low or negative when predictions explain little variation or perform poorly relative to the reference.

Model	MAE	RMSE	R^2	Observation
Linear Regression	0.890078	1.298526	0.150466	Lowest MAE in the recorded comparison.
Random Forest	0.950579	1.325256	0.115131	Highest MAE and RMSE among these three runs.
XGBoost	0.910882	1.284785	0.168350	Lowest RMSE and highest R^2 in the recorded comparison.

The metrics show a trade-off rather than a single universal winner. Linear Regression has the lowest MAE, which means it has the smallest average absolute error in this run. XGBoost has the lowest RMSE and highest R^2, suggesting a better result under those two measures. All three R^2 values are relatively modest, so the models explain only a limited share of the variation in the evaluation sample. These results should be reported honestly and used as a baseline for further improvement.

9.3 Interpretation of Results

The low average demand and presence of occasional higher values may make accurate prediction difficult. A simple model can perform well on many low-demand observations but still miss demand spikes. Conversely, a model

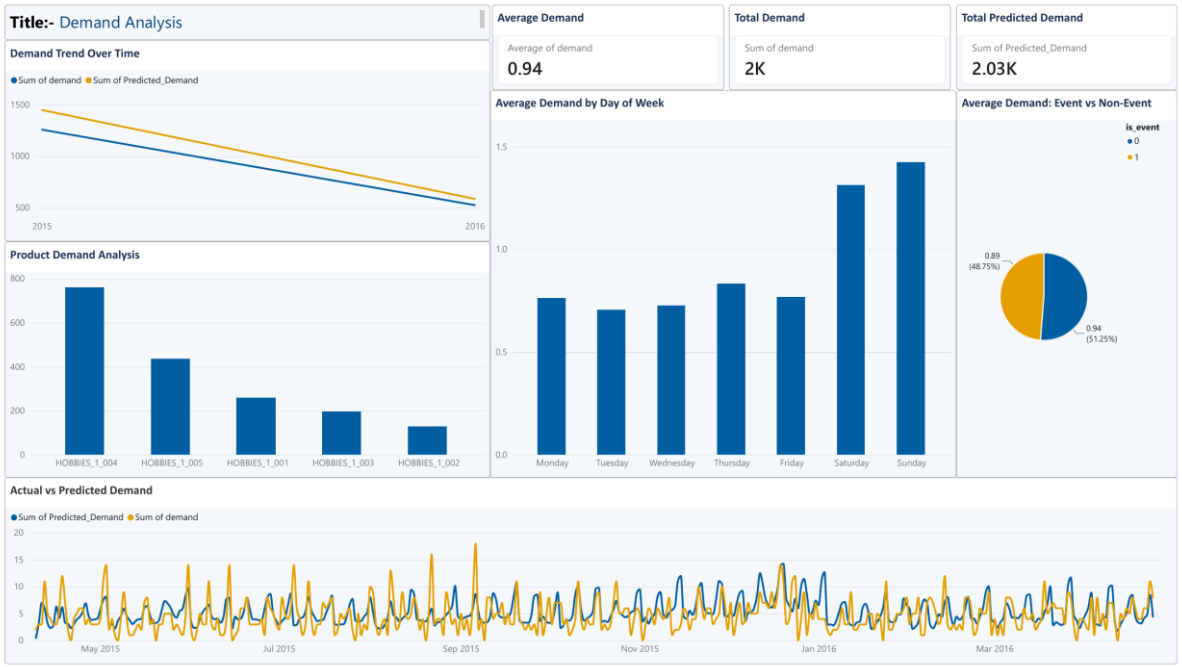
that attempts to capture peaks may create larger errors on quiet days. The appropriate model therefore depends on the business objective and the relative costs of under-forecasting and over-forecasting, not just one metric.

9.4 Power BI Dashboard 1 — Demand Analysis

The first dashboard focuses on understanding historical demand. It is intended to answer questions such as how demand changes over time, which selected products have higher demand, and whether weekday patterns are visible. A date or weekday slicer can narrow the analysis. Product-level charts should be interpreted within the scope of the five selected HOBBIES products and the CA_1 store represented in the current export.

A demand trend chart helps reveal broad increases or decreases. A weekday comparison can highlight differences between days, but weekday labels must be sorted by a numeric weekday order rather than alphabetically. Product comparisons provide a quick view of relative demand in the selected subset. Filters should be clearly labelled so that the audience knows which products, dates, and store are included.

Figure 8.1. Power BI Dashboard 1 — Demand Analysis



9.5 Power BI Dashboard 2 — Forecast and Inventory Insights

The second dashboard emphasizes actual demand versus predicted demand and error-related measures. A line chart or clustered column chart can compare actual and predicted values over time or by product. Error cards and tables can help identify periods where predictions diverge from actual observations. Inventory-oriented visuals should be described as indicators for investigation, not as direct proof that a stock-out occurred, unless inventory and availability records are also available.

Useful questions for this page include: Which products have the largest absolute forecast errors? In which periods are predictions consistently below actual demand? Do errors appear larger during event periods? Are the forecast differences small enough for the intended planning use? These questions support model review and help identify where more data or a different approach may be needed.

Figure 8.2. Power BI Dashboard 2 — Forecast and Inventory Insights



Dashboard interpretation: the visuals provide a consolidated view of the indicators available in this project. Use the page title and chart labels to interpret

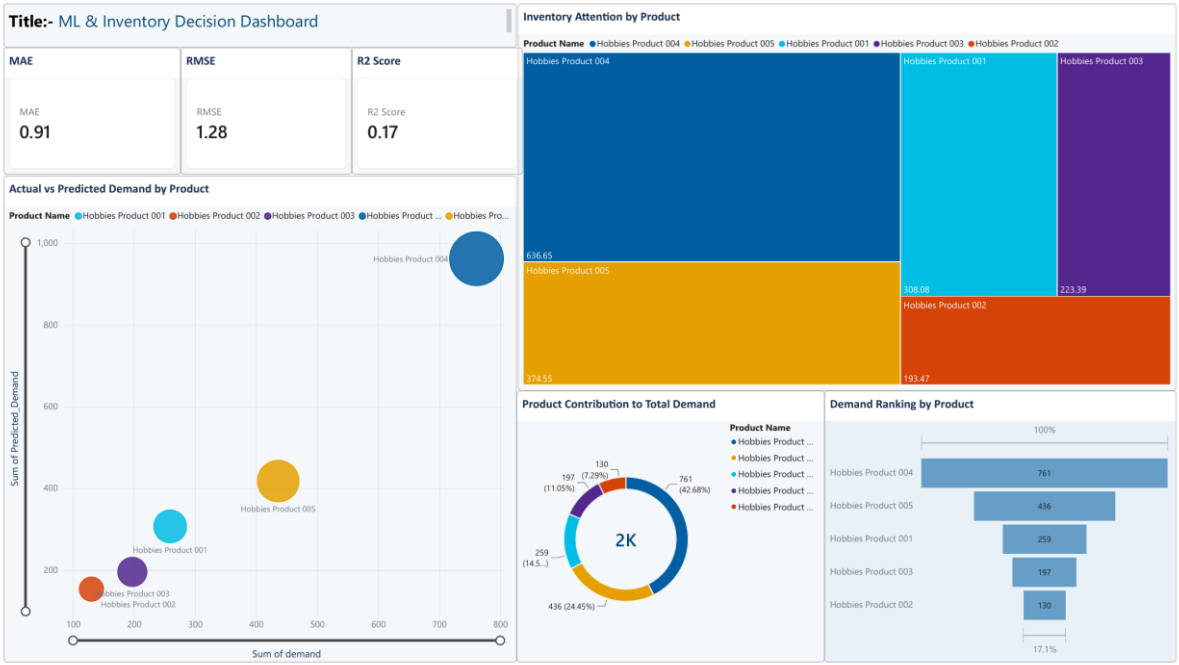
the measure being displayed, and apply product/date filters where available. The figures are reproduced from the submitted Power BI report; values should be interpreted within the current dataset scope and should not be generalised to other stores or product categories.

9.6 Power BI Dashboard 3 — ML and Inventory Decision Dashboard

The third dashboard brings model evaluation and decision-support indicators together. It can display MAE, RMSE, and R^2 for the tested models, alongside demand summaries and selected forecast outputs. Its purpose is to help the viewer understand the relative strengths and weaknesses of the models and avoid treating predictions as certain outcomes.

A decision-support page can also include a clear warning that recommendations depend on data quality, current stock, replenishment lead time, supplier constraints, and the business cost of different errors. If current inventory and lead-time fields are not present in the dataset, the dashboard should not label a computed value as a definitive reorder quantity.

Figure 8.3. Power BI Dashboard 3 — ML and Inventory Decision Dashboard



Dashboard interpretation: the visuals provide a consolidated view of the indicators available in this project. Use the page title and chart labels to interpret the measure being displayed, and apply product/date filters where available. The figures are reproduced from the submitted Power BI report; values should be interpreted within the current dataset scope and should not be generalised to other stores or product categories.

9.7 Suggested Advanced Business Questions

1. How does demand vary by day of the week and over time?
2. Which products have the highest average demand in the selected dataset?
3. How closely does predicted demand follow actual demand?
4. Which products or dates have the largest absolute forecast error?
5. Which tested model provides the most suitable balance of MAE, RMSE, and R^2 for the intended use?

9.8 Testing and Validation Checklist

- Confirm all required source files load successfully.
- Check row and column counts before and after reshaping and merging.
- Inspect missing values and duplicate keys.
- Verify that date fields have valid date types and chronological ordering.
- Confirm lag and rolling features do not use future or target-date demand.
- Check that training and evaluation data are separated consistently.
- Recalculate the metrics from saved predictions to confirm the reported values.
- Compare Power BI totals with the source CSV to detect import or aggregation issues.
- Verify weekday sorting and all slicer interactions.

- Document dataset scope, model version, and date range on the report page.

9.9 Extended Business Interpretation

The first dashboard, Demand Analysis, is intended for a quick understanding of the overall demand profile. The KPI cards summarise average demand, total demand, and total predicted demand. The time-series visual supports comparison of actual and predicted demand across the visible period. The weekday visual can help identify recurring day-level variation, while the event comparison provides a first look at differences between event and non-event observations. These patterns are exploratory and should be confirmed with additional periods before operational rules are introduced.

The second dashboard, Forecast and Inventory Insights, shifts attention from total volume to the behaviour of forecast errors and demand stability. Forecast error over time can expose periods in which the model overestimates or underestimates demand. Average demand by month and actual demand compared with the seven-day rolling measure provide context about short-term changes. The price-versus-demand visual is useful for exploring possible relationships, but a scatter plot alone does not establish that price changes caused demand changes.

The third dashboard, ML and Inventory Decision Dashboard, brings model metrics and product-level comparisons together. The reported MAE is approximately 0.91, RMSE approximately 1.28, and R-squared approximately 0.17 for the model view shown. MAE expresses the typical absolute prediction error in demand units; RMSE gives larger errors more influence; R-squared describes the share of variation explained relative to a simple mean baseline. These results indicate that the current model captures some patterns but leaves substantial variation unexplained. It should therefore be treated as a decision-support prototype rather than a fully reliable automated replenishment system.

An interactive review sequence for a store analyst is: (1) select the required product, (2) select a time window, (3) compare actual and predicted demand, (4)

inspect forecast-error periods, (5) compare the product with other products in the displayed set, and (6) use the evidence to flag an item for manual inventory review. The user should also check data freshness, price availability, and whether the selected product has enough historical observations. Any suggested replenishment quantity should consider lead time, current stock, minimum order quantities, shelf life, and service-level targets, which are not fully represented in the current prototype.

The report currently includes five hobby products and one store in the dashboard extract. Product ranking and contribution results are therefore relative only to those five products and that store. The dashboard does not support a valid comparison among multiple stores in its current form. A future version should include more products, additional stores, and a longer period of consistently cleaned data before management uses it for broad business conclusions.

9.10 Output Summary

The project demonstrates the full process from source data to model comparison and interactive reporting. The strongest result is the completed workflow and transparent comparison of alternatives. The recorded metrics suggest that the current models have limited predictive strength, so the project should be presented as a prototype and a foundation for further experimentation rather than as a finished commercial forecasting engine.

10. CONCLUSION

This mini-project explored demand forecasting for a quick-commerce store using machine learning. Historical sales were transformed into date-level observations, joined with calendar and selling-price information, cleaned, and enriched with time-based and demand-history features. Linear Regression, Random Forest, and XGBoost were trained and compared using MAE, RMSE, and R^2. The recorded experiment showed that Linear Regression achieved the

lowest MAE, while XGBoost achieved the lowest RMSE and the highest R^2 among the three tested models.

The project also used Power BI to present demand patterns, forecast comparisons, and model-related indicators in interactive dashboards. This reporting layer is important because forecasts are more useful when users can inspect the patterns and limitations behind them. The project provided practical experience in data preparation, feature engineering, supervised learning, evaluation, and dashboard development.

The findings must be interpreted with care. The model scores are modest, the current dashboard export covers a limited product-store subset, and the implemented feature set does not establish that all potentially useful external signals are included. The forecasts should therefore not be used as the sole basis for live inventory decisions. More rigorous time-based validation, broader data coverage, stronger baselines, and business-cost evaluation are necessary before operational deployment.

In conclusion, the project meets its learning objective by building an end-to-end demand forecasting prototype and communicating its results. It establishes a foundation for future work involving multi-store data, improved time-series validation, richer features, inventory-aware evaluation, and a controlled application interface for exploring predictions.